

文件系统

Joe

January 20, 2026

目录

1	简介	1
2	MBR 分区	1
2.0.1	MBR 分区方式	1
2.0.2	MBL 代码	2
2.0.3	磁盘签名	3
2.0.4	DPT 磁盘分区表	3
3	GPT 分区	7
4	FAT	7
4.1	FAT 布局	7
4.1.1	DBR	8
4.1.2	保留扇区	13
4.1.3	FAT 表	14
4.1.4	FAT 根目录	15
4.2	文件在哪里	16
4.2.1	目录项	17
4.2.2	文件磁盘空间分配	19
4.3	实现原理	20
4.3.1	文件创建	20
4.3.2	文件增删数据	20
4.3.3	文件数据读取	20
4.3.4	文件删除	21
4.3.5	基本原理	21

表格目录

1	文件系统术语	1
2	分区表字段含义	4
3	FAT16 记录项定义	14
4	FAT16 目录项 32 个字节定义	15
5	FAT32 簇定义	19

Listings

1 简介

文件系统的一些术语如表1所示。

表 1: 文件系统术语

缩写	英文	中文
MBR	Master Boot Record	主引导记录
GPT	GUID Partition Table	全局唯一标识符分区表
DPT	Disk Partition Table	磁盘分区表
MBL	Master Boot Loader	主引导加载程序
UEFI	Unified Extension Firmware Interface	统一可扩展固件接口
EBR	Extended Boot Record	扩展引导记录

- 主引导扇区: 是磁盘 0 号柱面,0 号磁头,1 号扇区,共 512Byte,其记录叫 MBR;
- $MBR = MBL(0 \sim 0x1BD = 446 \text{ 字节}) + DPT(4 \text{ 个主分区} * 16\text{Byte} = 64\text{Byte}) + "55AA" = 512\text{Byte}$;
- 主分区: 是由主引导扇区中 DPT 分区表 64 字节所定义,最多有 4 个主分区;
- 扩展分区: 为了满足更多分区需求,便产生了扩展分区,如果产生一个扩展分区,就必须牺牲一个主分区,扩展分区并不可以直接使用。
- 逻辑分区: 扩展分区必须以逻辑分区形式出现,即扩展分区包含了若干逻辑分区,至少包含 1 个逻辑分区,扩展分区中的逻辑分区是以链式存在。每个逻辑分区都记录着下个逻辑分区的位置信息,一次串联。每个逻辑分区都有一个类似主引导扇区的引导扇区,也有类似的分区表。该分区表记录了该分区的信息和一个指针,指向下个逻辑分区的引导扇区。由此,一旦某个逻辑分区损坏,则跟在它后面的逻辑分区都将丢失。
- 扇区: 逻辑上可操作的最小单位,一般都设置为 512Byte。
- 簇: 文件系统按一定数目的扇区为单位进行划分,这样的单位就是簇,通常情况下,每扇区 512 字节是不变的。簇的大小一般为 2^n 个扇区的大小。如 512B/1K/2K/4K/8K/16K/32K/64K。之所以以簇为单位而不是扇区进行磁盘分配,是因为当分区容量较大时,采用 512B 的扇区管理会增加 FAT 表的项数不利于大文件的存取,文件系统效率不高。FAT 文件系统之所以有 12/16/32 不同的版本之分,其根本在于 FAT 表用来记录任意 1 簇的二进制位数,如 FAT16,每 1 簇在 FAT 表中占据 2 字节,所以 FAT16 最大可以表示簇号为 0xFFFF,如果 32K 为簇,则 FAT16 可以管理的最大磁盘空间是: $32K * 0xFFFF = 2048MB = 2GB$,这就是为什么 FAT16 不支持超过 2G 分区的原因。

2 MBR 分区

PC 机上的分区方式主要有 MBR、GPT 和动态磁盘卷三种,用得比较多的是 MBR 和 GPT 分区方式。

2.0.1 MBR 分区方式

MBR 分区的数据信息是存储在磁盘的第一个扇区,也就是第 0 扇区,存储设备一个扇区大小一般为 512Byte。实际使用 MBR 方式分区,在没有扩展分区的条件下,分区只需要操作存储设备的第 0 扇区 512 个字节就可以了。通过 winhex 我们可以看到第 0 扇区的详细信息,如图1所示。

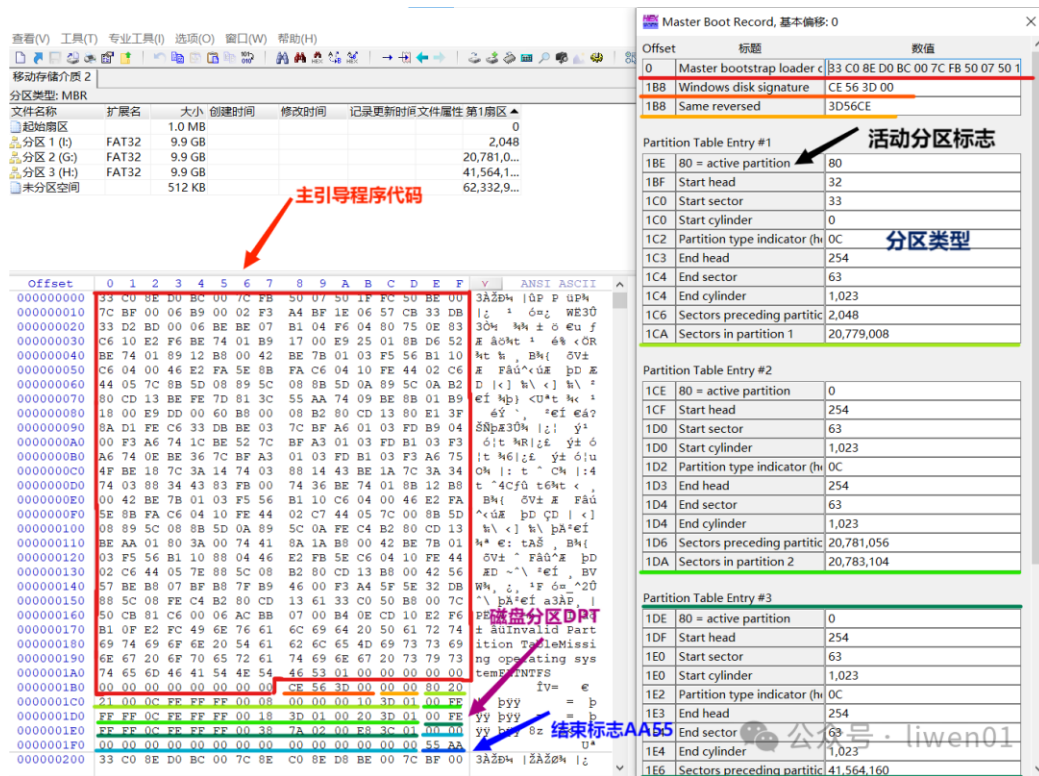


图 1: MBR 分区数据

它主要分为 4 部分.

- MBL 主引导程序代码 (440Bytes)
- 磁盘签名 (4Byte)
- DPT 分区表 (4*16Byte=64Byte)
- 结束标志 (55AAH)

2.0.2 MBL 代码

MBL 引导程序占用其中的前 440 字节, 其地址在偏移 0~0x1B7. 首先需要清楚的一点是, 这 440 个字节是一段程序, 这段程序的功能如下, 如图2所示.

- 先将第 0 扇区的 512 字节复制到内存的一个安全区域去执行;
- 判断第 0 扇区的最后两个字节是否为"55 AA", 如果不是, 提示出错;
- 查找 DPT 分区表中是否有活动分区 (0x80 标志);
- 如果有活动分区, 判断活动分区所在的扇区位置 (Start Sector);
- 将活动分区的引导扇区读入内存中, 并判断数据是否合法;
- 如果活动扇区数据合法, 就将引导权交给活动扇区;
- 活动扇区去引导操作系统启动, MBR 引导程序退出;

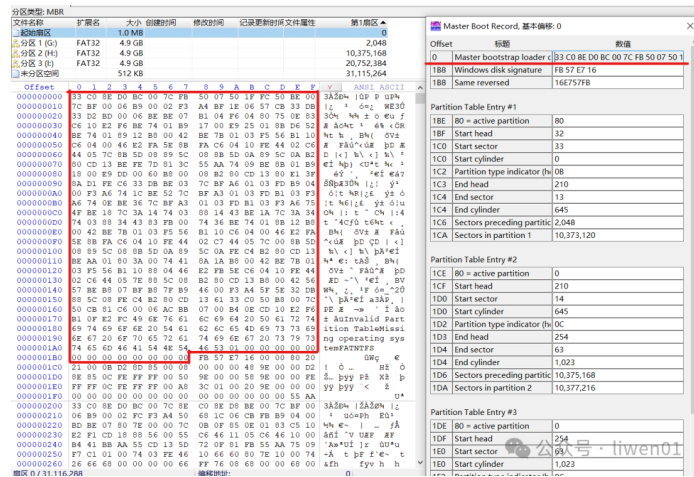


图 2: MBL 引导代码

活动分区: 一个磁盘中只能有一个活动扇区, 活动分区中包含引导加载程序, 它负责引导计算机启动操作系统. 活动分区是通过分区表项的第一个字节来判断, 分区表项的第一个字节, 只能是 0x00(非活动分区) 或者 0x80(活动分区).

2.0.3 磁盘签名

它是磁盘格式化时写入的标签, 如果把这个位置的数据清除, 系统会提示磁盘未初始化, 如图2所示的 0x1B8 位置的 4 字节数据“FB57E716”.

2.0.4 DPT 磁盘分区表

DPT 分区表结构如图3所示.

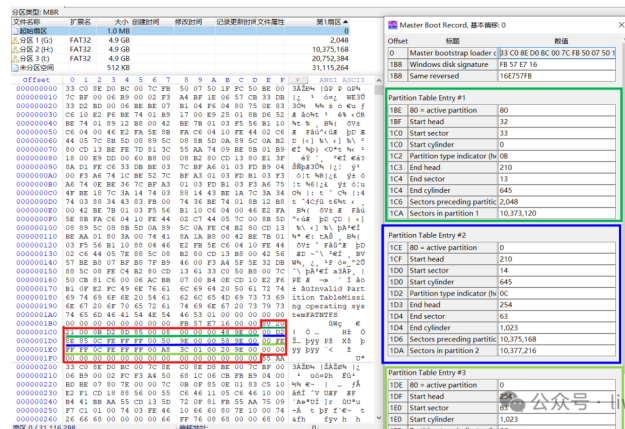


图 3: DPT 分区表数据

- MBR 磁盘分区表的结构是固定的, 总共包含 4 个表项, 每个表项 16 个字节.
- 每个表项对应一个主分区, 其中可以存储主分区或者扩展分区的信息.
- 如果不使用扩展分区, 一个磁盘最多就只能支持 4 个分区

DPT 磁盘分区表有如下概念, 其定义如表2所示.

表 2: 分区表字段含义

Offset	Size	Value	Description
0x01BE	1Byte	0x80	引导指示符, 指明是否活动分区
0x01BF	1Byte	0x20	开始磁头 (Start head)
0x01C0	6bit	0x21	开始扇区 (Start sector)
0x01C1	10bit	0x00	开始柱面 (Start Cylinder),
0x01C2	1Byte	0x0B	系统 ID 或分区类型 (Partition type indicator)
0x01C3	1Byte	0xD2	结束磁头 (End Head)
0x01C4	6bit	0x0D	结束扇区 (End Sector)
0x01C5	10bit	0x285	结束柱面 (End Cylinder)
0x01C6	4Byte	0x00000800	相对扇区数 (Relative Sector)
0x01CA	4Byte	0x009E4800	总扇区数 (Total Sector)

- Start/End sector: 6bit, 这个字节只用 6bit, bit6/7 被下个字段所用, 所以一个盘面, 只有 64 个 sector, 每个 sector, 512byte.
- Start/End Cylinder: 10bit, 柱面共 1024 个, 实际上用这种方式表示的分区容量是有限的, 柱面和磁头从 0 开始编号, 扇区从 1 开始编号, 所以最多只能表示 1024 柱面 * 63 个扇区 * 256 个磁头 * 512byte = 8455716864byte = 8.4GB (实际上应该是 7.8GB 左右), 实际磁头数通常只用到 255 个 (汇编语言的寻址寄存器决定).
- Relative sector: 相对扇区数, 从磁盘的开始到该分区的开始的位移量, 以扇区来计算, 如表中 0x800, 即 Offset 为 2048 个扇区.
- Total Sector: 该分区种的扇区总数.
- 分区表项格式: 每个分区表项包含了有关分区的信息, 包括分区的起始位置、大小、文件系统类型等. 此外, 每个分区表项还包含一个用于标识分区状态 (活动/非活动) 的标志.
- 活动分区: 活动分区是一个特殊的主分区, 被标记为活动. 只有活动分区上的引导加载程序才会被计算机 BIOS 或 UEFI 加载, 从而启动操作系统.
- 结束标志: 当计算机引导时, BIOS 或 UEFI 会检查这个结束标志, 以确认 MBR 分区表的有效性. 如果这个标志不存在或不等于 0x55AA, 计算机可能无法正确识别和引导操作系统.
- 主分区: MBR 分区表最多支持 4 个主分区. 如果需要更多的分区, 可以使用其中一个主分区作为扩展分区, 然后在扩展分区内创建逻辑分区. 逻辑分区的信息存储在扩展分区的扩展分区记录中.
- 扩展分区: 严格地讲它不是一个实际意义的分区, 它仅仅是一个指向下一个用来定义分区的参数的指针, 这种指针结构形成一个单向链表. 通过 MBR 上的扩展分区, 逐个遍历就可以找到磁盘上的所有扩展分区.

将一个 16G 的 U 盘分成 4 个分区, 两个主分区, 两个扩展分区, 分区参数如图 4 所示.



图 4: U 盘分区

它的在磁盘中的数据结构如图5所示。

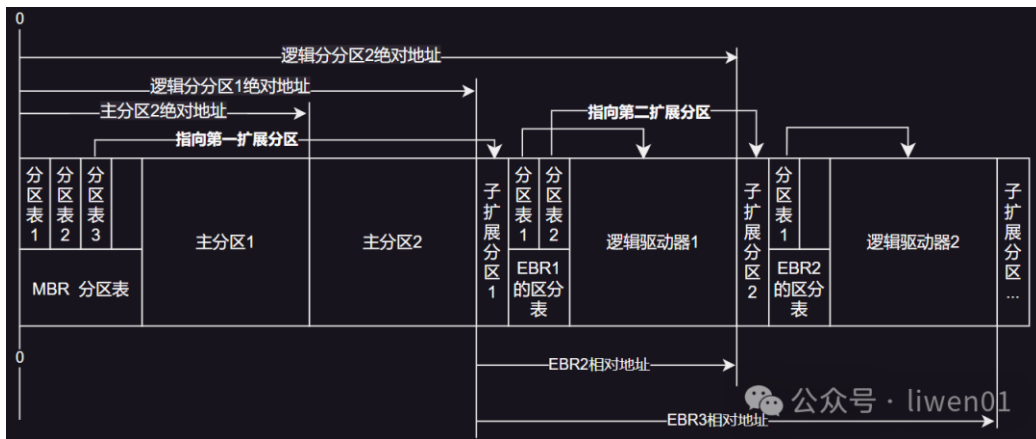


图 5: U 盘分区格式

有几个比较重要的点。

- MBR 理论可以放置 4 个分区表项,实际只使用了 3 个;
- MBR 的第三分区表项为逻辑分区,它指向第一扩展分区的位置;
- 扩展分区的分区表里只有两个有效分区表项,一个指向自己,另外一个指向下一个扩展分区;
- 如果是最后一个扩展分区,它可以只有一个分区表项;
- 在逻辑分区中,它们使用的是相对地址,相对于第一扩展分区的地址,而不是使用绝对地址;

可以查看第 0 扇区数据进行核对,第 4 分区表项位置数据全是空,另外第一二分区类型为 0x0B 为 FAT32 分区,第三分区类型为 0x0F,表示为扩展分区,如图 7 所示,定义如图 6 所示。

分区类型标志:	
00 空, microsoft不允许使用.	63 GNU HURD or Sys
01 FAT32	64 Novell Netware
02 XENIX root	65 Novell Netware
03 XENIX usr	70 Disk Secure Mult
04 FAT16 <32M	75 PC/IX
05 Extended	80 Old Minix
06 FAT16	81 Minix/Old Linux
07 HPFS/NTFS	82 Linux swap
08 AIX	83 Linux
09 AIX bootable	84 OS/2 hidden C:
0A OS/2 Boot Manage	85 Linux extended
0B Win95 FAT32	86 NTFS volume set
0C Win95 FAT32	87 NTFS volume set
0E Win95 FAT16	93 Amoeba
0F Win95 Extended(>8GB)	94 Amoeba BBT
10 OPUS	A0 IBM Thinkpad hidden
11 Hidden FAT12	A5 BSD/386
12 Compaq diagnost	A6 Open BSD
16 HiddenFAT16	A7 NextSTEP
14 Hidden FAT16<32GB	B7 BSDI fs
17 Hidden HPFS/NTFS	B8 BSDI swap
18 AST Windows swap	BE Solaris boot
1B Hidden FAT32	partition
1C Hidden FAT32 partition	C0 DR-DOS/Novell DOS
(using LBA-mode	secured partition
INT 13 extensions)	C1 DRDOS/sec
1E Hidden LBA VFAT partition	C4 DRDOS/sec
24 NEC DOS	C6 DRDOS/sec
3C Partition Magic	C7 Syrix
40 Venix 80286	DB CP/M/CTOS
41 PPC PreP Boot	E1 DOS access
42 SFS	E3 DOS R/O
4D QNX4.x	E4 SpeedStor
4E QNX4.x 2nd part	EB BeOS fs
4F QNX4.x 3rd part	F1 SpeedStor
50 Ontrack DM	F2 DOS 3.3+ secondary
51 Ontrack DM6 Aux	partition
52 CP/M	F4 SpeedStor
53 onTRACK DM6 Aux	FE LAN step
54 OnTrack DM6	FF BBT
55 EZ-Drive	
56 Golden Bow	
5C Priam Edisk	
61 Speed Stor	

www.sjhf.net
图4

图 6: 分区表类型定义

- Windows 常见的类型标志有: 0x00/0x01/0x04/0x05/0x07/0x0B/0x0C/0x0E/0x42/0x80;
- Linux 常见的类型标志有: 0x82/0x83/0x85/0xa50/0xa6;
- 其他操作系统类型标志有: 0xA0/0xBE/0xEB;

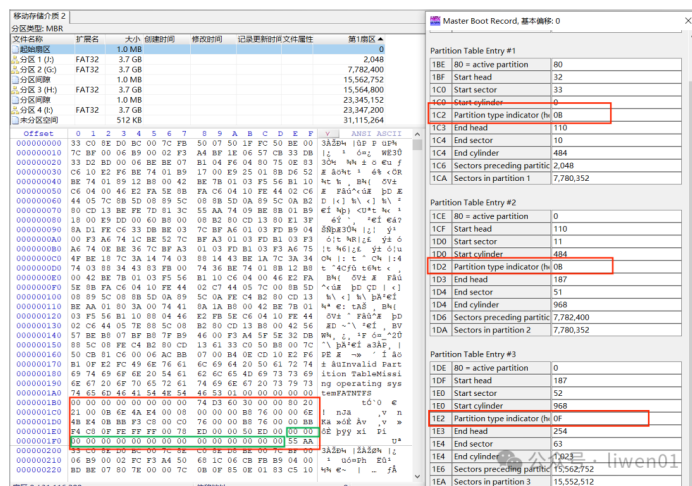


图 7: 分区类型图

MBR 分区的局限性,MBR 是一种比较老旧的分区格式,它有更多的局限性.

- 最多支持 4 个主分区,MBR 分区表最多支持 4 个主分区,其中一个可以是扩展分区;
- 扩展分区和逻辑分区结构复杂,MBR 通过扩展分区来克服主分区数量的限制,但使用扩展分区和逻辑分区引入了一些复杂性;
- 2TB 以上硬盘的限制,MBR 使用 32 位的逻辑块地址 (LBA) 来表示分区的起始位置和大小,因此存在 2TB(2^{41} 字节) 的硬盘大小限制;
- 不支持 UEFI 启动,MBR 是基于传统 BIOS 引导的,不支持新一代的 UEFI 引导方式;
- 不具备数据完整性校验,MBR 分区表中没有内置的数据完整性校验机制,因此在一些情况下,分区表损坏或错误可能导致数据丢失或系统启动问题;

3 GPT 分区

4 FAT

FAT32 是从 FAT12、FAT16 发展而来,目前主要应用在移动存储设备中,比如 SD 卡、TF 卡. 隐藏的 FAT 文件系统现在也有被大量使用在 UEFI 启动分区中.

4.1 FAT 布局

拿一个 FAT32 文件系统的存储设备,可以看到,它整个存储设备大概可以分为 5 个部分,如图8所示.



图 8: FAT32 磁盘布局

- 引导和保留扇区: 会因为分区方式的不同 (MBR 与 GPT) 而不同,同时也会因为存储设备分区个数的不同也会有差异. 图9这个是使用 GPT 方式将存储设备分为 1 个分区并格式化为 FAT32 文件系统格式的数据分布示意图.

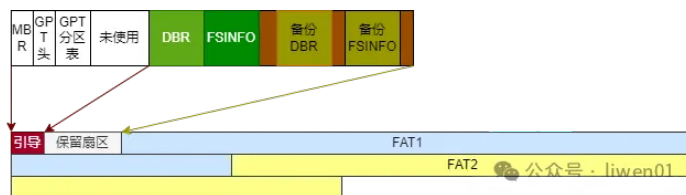


图 9: FAT32 GPT 分区方式

- FAT 表和目录/文件: 在 FAT32 文件系统的使用过程中, FAT 表和目录项是其核心部分;
- 备份: 在磁盘末尾的备份区域, 主要是使用 GPT 方式分区的时候, 会将分区表信息备份到存储设备的最后区域. 对于 MBR 分区方式, 并没有这部分.

FAT 文件系统还有一种视角, 如图10所示, FAT 分区格式是 Microsoft 最早提出的分区格式. 依据 FAT 表中每个簇链的所占位数分为 FAT12/FAT16/FAT32 三种格式.

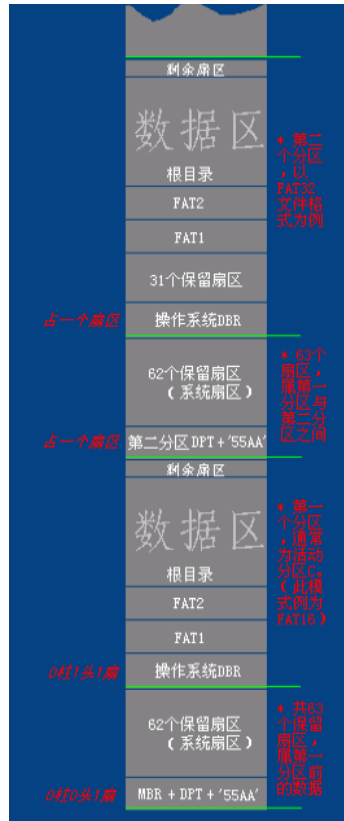


图 10: FAT32 分区结构视角

4.1.1 DBR

DBR 区 (DOC Boot Record) 即操作系统引导记录区, 通常占用该分区的第 0 扇区共 512 字节 (特殊情况下也会占用其他保留扇区). 如图11所示是一个 DBR 数据示例.

Offset	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	对应字符
00000000	EB	58	90	4D	53	57	49	4E	34	2E	31	00	02	08	20	00	隔恣 SWIN4.1... .
00000010	02	00	00	00	00	F8	00	00	3F	00	FF	00	3F	00	00	00	?.?. ??...
00000020	3F	04	7D	00	32	1F	00	00	00	00	00	00	02	00	00	00	?..2.....
00000030	01	00	06	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000040	80	00	29	FE	1C	39	33	4E	4F	20	4E	41	4D	45	20	20	ε.)?93NO NAME
00000050	20	20	46	41	54	33	32	20	20	20	FA	33	C9	8E	D1	BC	FAT32 ?麾鸭
00000060	F8	7B	8E	C1	BD	78	00	C5	76	00	1E	56	16	55	BF	22	鸚鵡經.狀..V.U?
00000070	05	89	7E	00	89	4E	02	B1	0B	FC	F3	A4	8E	D9	BD	00	.墟.墟.? 伟.
00000080	7C	C6	45	FE	0F	8B	46	18	88	45	F9	38	4E	40	7D	25	[拏?婢.圖?N@)%
00000090	8B	C1	99	EB	00	07	E8	97	00	72	1A	83	EB	3A	66	A1	婆嶺..枿.r.泮:f?
000000A0	1C	7C	66	3B	07	8A	57	FC	75	06	80	CA	02	88	56	02	. f;.奧驢.e?均.
000000B0	80	C3	10	73	ED	BF	02	00	83	7E	16	00	75	45	8B	46	e?s枫..僂..uE婢
000000C0	1C	8B	56	1E	B9	03	00	49	40	75	01	42	BB	00	7E	E8	.媽.?..I@u.B?~?
000000D0	5F	00	73	26	B0	F8	4F	74	1D	8B	46	32	33	D2	B9	03	..s&傍Ot.婢.23夜
000000E0	00	3B	C8	77	1E	8B	76	0E	3B	CE	73	17	2B	F1	03	46	.;墓.燥.;蜈.+?F
000000F0	1C	13	56	1E	EB	D1	73	0B	EB	27	83	7E	2A	00	77	03	..V.胙.s.?僂*.w.
00000100	E9	FD	02	BE	7E	7D	AC	98	03	F0	AC	84	C0	74	17	3C	羣.縑}璫.腿勃t.<
00000110	FF	74	09	B4	0E	BB	07	00	CD	10	EB	EE	BE	81	7D	EB	t.???脯縵}?
00000120	E5	BE	7F	7D	EB	E0	98	CD	16	5E	1F	66	8F	04	CD	19	竇 }豚穆.^..f??
00000130	41	56	66	6A	00	52	50	06	53	6A	01	6A	10	8B	F4	60	AVfj.RP.Sj.j.嫫
00000140	80	7E	02	0E	75	04	B4	42	EB	1D	91	92	33	D2	F7	76	e~..u.簪?嫫3吟v
00000150	18	91	F7	76	18	42	87	CA	F7	76	1A	8A	F2	8A	E8	C0	.尻v.B轉縵.嫫嫫
00000160	CC	02	0A	CC	B8	01	02	8A	56	40	CD	13	61	8D	64	10	?..談..葵@?a崗.
00000170	5E	72	0A	40	75	01	42	03	5E	0B	49	75	B4	C3	0D	0A	^r.@u.B.^..Iu縵..
00000180	49	6E	76	61	6C	69	64	20	73	79	73	74	65	6D	20	64	Invalid system d
00000190	69	73	6B	0D	0A	44	69	73	6B	20	49	2F	4F	20	65	72	isk..Disk I/O er
000001A0	72	6F	72	0D	0A	52	65	70	6C	61	63	65	20	74	68	65	ror..Replace the
000001B0	20	64	69	73	6B	2C	20	61	6E	64	20	74	68	65	6E	20	disk, and then
000001C0	70	72	65	73	73	20	61	6E	79	20	6B	65	79	0D	0A	00	press any key...
000001D0	6B	65	79	0D	0A	00	00	00	49	4F	20	20	20	20	20	20	key.....IO
000001E0	53	59	53	4D	53	44	4F	53	20	20	20	59	59	53	7E	01	SYSMSDOS SYS~.
000001F0	00	57	49	4E	42	4F	4F	54	20	53	59	53	00	00	55	AA	.WINBOOT SYS..U

图 11: FAT32 DBR 数据

DBR 数据各字段解析如图12所示.

字节位移	字段长度	字段名	对应图 8 颜色
0x00	3 个字节	跳转指令	
0x03	8 个字节	厂商标志 和 os 版本号	
0x0B	53 个字节	BPB	
0x40	26 个字节	扩展 BPB	
0x5A	420 个字节	引导程序 代码	
0x01FE	2 个字节	有效结束 标志	

图 12: FAT32 DBR 解析

- 跳转指令;
- 厂商标志;
- 操作系统版本号;
- BPB(Bios Parameter Block);
- 扩展 BPB;
- OS 引导程序;

- 结束标志.

进一步解析如图13所示.

Boot Sector FAT32, 基础偏移量: 0		
Offset	标题	数值
0	JMP instruction	EB 58 90
3	OEM	MSWIN4.1
BIOS Parameter Block		
B	Bytes per sector	512
D	Sectors per cluster	8
E	Reserved sectors	32
10	Number of FATs	2
11	Root entries (unused)	0
13	Sectors (on small volumes)	0
15	Media descriptor (hex)	F8
16	Sectors per FAT (small vol.)	0
18	Sectors per track	63
1A	Heads	255
1C	Hidden sectors	63
20	Sectors (on large volumes)	8193087
FAT32 Section		
24	Sectors per FAT	7986
28	Flags	0
2A	Version	0
2C	Root dir 1st cluster	2
30	FSInfo sector	1
32	Backup boot sector	6
34	(Reserved)	00 00 00 00 00 00 00 00 00 00 00 00
40	BIOS drive (hex, HD=8x)	80
41	(Unused)	0
42	Ext. boot signature (29h)	29
43	Volume serial number (decimal)	859380990
43	Volume serial number (hex)	FE 1C 39 33
47	Volume label	NO NAME
52	File system	FAT32
1FE	Signature (55 AA)	55 AA

图 13: FAT32 DBR 详细解析

- JMP instruction: MBR 将 CPU 执行转移给引导扇区, 由此, 引导扇区的前 3 个字节必须是合法的基于 x86/ARM/RISCV 的 CPU 指令, 通常是一条跳转指令, 该指令负责跳过接下来的几个不可执行的字节 (BPB 和扩展 BPB), 跳到 OS 引导程序部分.
- OEM: 8 字节长的 OEM ID, 一个字符串, 它标识了格式化该分区的操作系统的名称和版本号.
 - 为了保留与 DOS 的兼容性, 通常 Win2000 格式化该盘是在 FAT16 和 FAT32 磁盘上的该字段记录了"MSDOS5.0";
 - 在 NTFS 磁盘上, Win2000 记录的是"NTFS";
 - 在被 Win95 格式化的是"MSWIN4.0";
 - 在被 Win95 OSR2 和 Win98 格式化的是"MSWIN4.1".

- BPB 区 (Bios Parameter Block), 下面以 (偏移, 长度, 值) 进行描述.
 - (0x0B, 2Byte, 0x0200): 扇区字节数 (BytesPerSector) 硬件扇区的大小. 本字段合法的十进制值有 512、1024、2048 和 4096. 对大多数磁盘来说, 本字段的值为 512;
 - (0x0D, 1Byte, 0x08): 每簇扇区数 (SectorsPerCluster), 一簇中的扇区数. 由于 FAT32 文件系统只能跟踪有限个簇 (最多为 4294967296 个), 因此, 通过增加每簇扇区数, 可以使 FAT32 文件系统支持最大分区数. 一个分区缺省的簇大小取决于该分区的大小. 本字段的合法十进制值有 1、2、4、8、16、32、64 和 128. Win2000 的 FAT32 实现只能创建最大为 32GB 的分区. 但是, Win2000 能够访问由其他操作系统 (Win95、OSR2 及其以后的版本) 所创建的更大的分区;
 - (0x0e, 2Byte, 0x0020): 保留扇区数 (ReservedSector) 第一个 FAT 开始之前的扇区数, 包括引导扇区. 本字段的十进制值一般为 32;
 - (0x10, 1Byte, 0x02): FAT 数 (Number of FAT) 该分区上 FAT 的副本数. 本字段的值一般为 2;
 - (0x11, 2Byte, 0x0000): 根目录项数 (RootEntries) 只有 FAT12/FAT16 使用此字段. 对 FAT32 分区而言, 本字段必须设置为 0;
 - (0x13, 2Byte, 0x0000): 小扇区数 (SmallSector)(只有 FAT12/FAT16 使用此字段) 对 FAT32 分区而言, 本字段必须设置为 0;
 - (0x15, 1Byte, 0xF8): 媒体描述符 (MediaDescriptor) 提供有关媒体被使用的信息. 值 0xF8 表示硬盘, 0xF0 表示高密度的 3.5 寸软盘. 媒体描述符要用于 MS-DOS FAT16 磁盘, 在 Win2000 中未被使用
 - (0x16, 2Byte, 0x0000): 每 FAT 扇区数 (SectorsPerFAT) 只被 FAT12/FAT16 所使用, 对 FAT32 分区而言, 本字段必须设置为 0
 - (0x18, 2Byte, 0x003F): 每道扇区数 (Sectors PerTrack) 包含使用 INT13h 的磁盘的“每道扇区数”几何结构值. 该分区被多个磁头的柱面分成了多个磁道;
 - (0x1A, 2Byte, 0x00FF): 磁头数 (Number of Head) 本字段包含使用 INT 13h 的磁盘的“磁头数”几何结构值. 例如, 在一张 1.44MB 3.5 英寸的软盘上, 本字段的值为 2;
 - (0x1C, 4Byte, 0x0000003F): 隐藏扇区数 (Hidden Sector) 该分区上引导扇区之前的扇区数. 在引导序列计算到根目录的数据区的绝对位移的过程中使用了该值. 本字段一般只对那些在中断 13h 上可见的媒体有意义. 在没有分区的媒体上它必须总是为 0;
 - (0x20, 4Byte, 0x007D043F): 总扇区数 (Large Sector) 本字段包含 FAT32 分区中总的扇区数;
 - (0x24, 4Byte, 0x00001F32): 每 FAT 扇区数 (Sectors PerFAT)(只被 FAT32 使用) 该分区每个 FAT 所占的扇区数. 计算机利用这个数和 FAT 数以及隐藏扇区数 (本表中所描述的) 来决定根目录从哪里开始. 该计算机还可以从目录中的项数决定该分区的用户数据区从哪里开始;
 - (0x28, 2Byte, 0x00): 扩展标志 (Extended Flag)(只被 FAT32 使用) 该两个字节结构中各位的值为:
 - 1. 位 0-3: 活动 FAT 数 (从 0 开始计数, 而不是 1). 只有在不使用镜像时才有效
 - 2. 位 4-6: 保留
 - 3. 位 7: 0 值意味着在运行时 FAT 被映射到所有的 FAT 1 值表示只有一个 FAT 是活动的
 - 4. 位 8-15: 保留
 - (0x2A, 2Byte, 0x0000): 文件系统版本 (File systemVersion) 只供 FAT32 使用, 高字节是主要的修订号, 而低字节是次要的修订号. 本字段支持将来对该 FAT32 媒体类型进行扩展. 如果本字段非零, 以前的 Windows 版本将不支持这样的分区;
 - (0x2C, 4Byte, 0x00000002): 根目录簇号 (Root ClusterNumber)(只供 FAT32 使用) 根目录第一簇的簇号. 本字段的值一般为 2, 但不总是如此;

- (0x30, 2Byte, 0x0001): 文件系统信息扇区号 (FileSystem InformationSectorNumber)(只供 FAT32 使用)FAT32 分区的保留区中的文件系统信息 (File SystemInformation, FSINFO) 结构的扇区号. 其值一般为 1. 在备份引导扇区 (Backup BootSector) 中保留了该 FSINFO 结构的一个副本,但是这个副本不保持更新;
- (0x34, 2Byte, 0x0006): 备份引导扇区 (只供 FAT32 使用) 为一个非零值,这个非零值表示该分区保存引导扇区的副本的保留区中的扇区号. 本字段的值一般为 6, 建议不要使用其他值;
- (0x36, 12Byte, 12 个字节均为 0x00): 保留 (只供 FAT32 使用) 供以后扩充使用的保留空间. 本字段的值总为 0;
- (0x40, 1Byte,0x80): 物理驱动器号 (PhysicalDriveNumber) 与 BIOS 物理驱动器号有关. 软盘驱动器被标识为 0x00, 物理硬盘被标识为 0x80, 而与物理磁盘驱动器无关. 一般地, 在发出一个 INT13h BIOS 调用之前设置该值, 具体指定所访问的设备. 只有当该设备是一个引导设备时, 这个值才有意义;
- (0x41, 1Byte,0x00): 保留 (Reserved) FAT32 分区总是将本字段的值设置为 0;
- (0x42, 1Byte, 0x29): 扩展引导标签 (ExtendedBoot Signature) 本字段必须要有能被 Win2000 所识别的值 0x28 或 0x29;
- (0x43, 4Byte, 0x33391CFE): 分区序号 (Volume SerialNumber) 在格式化磁盘时所产生的一个随机序号, 它有助于区分磁盘
- (0x47, 11Byte, "NO NAME"): 卷标 (Volume Label) 本字段只能使用一次, 它被用来保存卷标号. 现在, 卷标被作为一个特殊文件保存在根目录中
- (0x52, 8Byte, "FAT32"): 系统 ID(System ID)FAT32 文件系统中一般取为"FAT32";

DBR 的偏移 0x5A 开始的数据为操作系统引导代码. 这是由偏移 0x00 开始的跳转指令所指向的. 在图 11 所列出的偏移 0x00~0x02 的跳转指令"EB5890"清楚地指明了 OS 引导代码的偏移位置.jump 58H 加上跳转指令所需的位移量, 即开始于 0x5A. 此段指令在不同的操作系统上和不同的引导方式上, 其内容也是不同的. 大多数的资料上都说 win98, 构建于 fat 基本分区上的 Win2000, Winxp 所使用的 DBR 只占用基本分区的第 0 扇区. 他们提到, 对于 Fat32, 一般的 32 个基本分区保留扇区只有第 0 扇区是有用的. 实际上, 以 FAT32 构建的操作系统如果是 Win98, 系统会使用基本分区的第 0 扇区和第 2 扇区存储 os 引导代码; 以 FAT32 构建的操作系统如果是 Win2000 或 Winxp, 系统会使用基本分区的第 0 扇区和第 0xC 扇区 (Win2000 或 Winxp, 其第 0xC 的位置由第 0 扇区的 0xAB 偏移指出) 存储 os 引导代码. 所以, 在 Fat32 分区格式上, 如果 DBR 一扇区的内容正确而缺少第 2 扇区 (Win98 系统) 或第 0xC 扇区 (Win2000 或 Winxp 系统), 系统也是无法启动的. 如果自己手动设置 NTLDR 双系统, 必须知道这一点.

DBR 扇区的最后两个字节一般存储值为 0x55AA 的 DBR 有效标志, 对于其他的取值, 系统将不会执行 DBR 相关指令. 上面提到的其他几个参与 os 引导的扇区也需以 0x55AA 为合法结束标志.

4.1.2 保留扇区

在 FAT 文件系统 DBR 的偏移 0x0E 处, 用 2 个字节存储保留扇区的数目. 所谓保留扇区 (有时候会叫系统扇区, 隐藏扇区), 是指从分区 DBR 扇区开始的仅为系统所有的扇区, 包括 DBR 扇区. 在 FAT16 文件系统中, 保留扇区的数据通常设置为 1, 即仅仅 DBR 扇区. 而在 FAT32 中, 保留扇区的数据通常取为 32, 有时候用 PartitionMagic 分过的 FAT32 分区会设置 36 个保留扇区, 有的工具可能会设置 63 个保留扇区.

FAT32 中的保留扇区除了磁盘总第 0 扇区用作 DBR, 总第 2 扇区 (win98 系统) 或总第 0xC 扇区 (Win2000, Winxp) 用作 OS 引导代码扩展部分外, 其余扇区都不参与操作系统管理与磁盘数据管理, 通常情况下是没作用的. 操作系统之所以在 FAT32 中设置保留扇区, 是为了对 DBR 作备份或留待以后升级时用. FAT32 中, DBR 偏移 0x34 占 2 字节的数据指明了 DBR 备份扇区所在, 一般为 0x06, 即第 6

扇区。当 FAT32 分区 DBR 扇区被破坏导致分区无法访问时，可以用第 6 扇区的原备份替换第 0 扇区来找回数据。

4.1.3 FAT 表

FAT 表 (File Allocation Table 文件分配表)，是 Microsoft 在 FAT 文件系统中用于磁盘数据 (文件) 索引和定位引进的一种链式结构。假如把磁盘比作一本书，FAT 表可以认为相当于书中的目录，而文件就是各个章节的内容。但 FAT 表的表示方法却与目录有很大的不同。在 FAT 文件系统中，文件的存储依照 FAT 表制定的簇链式数据结构来进行。同时，FAT 文件系统将组织数据时使用的目录也抽象为文件，以简化对数据的管理。

FAT 表实际上是一个数据表，以 2 个字节为单位，我们暂将这个单位称为 FAT 记录项，通常情况其第 1、2 个记录项 (前 4 个字节) 用作介质描述。从第 3 个记录项开始记录除根目录外的其他文件及文件夹的簇链情况。根据簇的表现情况 FAT 用相应的取值来描述，以 FAT16 记录项为例，见表 3 所示的定义。

表 3: FAT16 记录项定义	
簇号	对应簇的表现情况
0x0000	未分配的簇
0x0002~0xFFEF	已分配的簇
0xFFFF0~0xFFFF6	系统保留
0xFFFF7	坏簇
0xFFFF8~0xFFFFF	文件结束簇

如图 14，FAT 表以“F8 FF FF FF”开头，此 2 字节为介质描述单元，并不参与 FAT 表簇链关系。

驱动器 L:

Offset	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	访问		
00000020	F8	FF	FF	FF	FF	2F	05	00	06	00	FF	5F	FF	07	00	08	70	?	
00000024	09	00	0A	00	0B	00	0C	00	0D	00	0E	00	0F	00	10	00			
00000028	11	00	12	00	13	00	14	00	15	00	16	00	17	00	18	00			
0000002C	19	00	1A	00	1B	00	1C	00	1D	00	1E	00	1F	00	20	00			
00000030	21	00	22	00	23	00	24	00	25	00	26	00	27	00	28	00	! " # \$ % & ' (.		
00000034	29	00	2A	00	2B	00	2C	00	2D	00	2E	00	2F	00	30	00	). * + , - . / 0 .		
00000038	31	00	32	00	33	00	34	00	35	00	36	00	37	00	38	00	1 . 2 . 3 . 4 . 5 . 6 . 7 . 8 .		
0000003C	39	00	3A	00	3B	00	3C	00	3D	00	3E	00	3F	00	40	00	9 . : . ; . < . = . > . ? . @ .		
00000040	41	00	42	00	43	00	44	00	45	00	46	00	47	00	48	00	A . B . C . D . E . F . G . H .		
00000044	49	00	4A	00	4B	00	4C	00	4D	00	4E	00	4F	00	50	00	I . J . K . L . M . N . O . P .		
00000048	51	00	52	00	53	00	54	00	55	00	56	00	57	00	58	00	Q . R . S . T . U . V . W . X .		
0000004C	59	00	5A	00	5B	00	5C	00	5D	00	5E	00	5F	00	60	00	Y . Z . [. \ .] . ^ . _ . ` .		
00000050	61	00	62	00	63	00	64	00	65	00	66	00	67	00	68	00	a . b . c . d . e . f . g . h .		
00000054	69	00	6A	00	6B	00	6C	00	6D	00	6E	00	6F	00	70	00	i . j . k . l . m . n . o . p .		
00000058	71	00	72	00	73	00	74	00	75	00	76	00	77	00	78	00	q . r . s . t . u . v . w . x .		
0000005C	79	00	7A	00	7B	00	7C	00	7D	00	7E	00	7F	00	80	00	y . z . { . . } . ~ . € .		
00000060	81	00	82	00	83	00	84	00	85	00	86	00	87	00	88	00	????????} . ~ . € .		
00000064	89	00	8A	00	8B	00	8C	00	8D	00	8E	00	8F	00	90	00	????????} . ~ . € .		
00000068	91	00	92	00	93	00	94	00	95	00	96	00	97	00	98	00	????????} . ~ . € .		
0000006C	FF	FF	9A	00	FF	FF	9C	00	9D	00	FF	FF	9F	00	A0	00	? ?? ?		
00000070	FF	FF	FF	FF	FF	FF	A4	00	A5	00	A6	00	A7	00	A8	00	?????		
00000074	A9	00	AA	00	AB	00	AC	00	AD	00	AE	00	AF	00	B0	00	???????		
00000078	B1	00	B2	00	B3	00	B4	00	B5	00	B6	00	B7	00	B8	00	???????		
0000007C	B9	00	BA	00	BB	00	BC	00	BD	00	BE	00	BF	00	C0	00	???????		
00000080	C1	00	C2	00	C3	00	C4	00	C5	00	C6	00	C7	00	C8	00	???????		

图 14: FAT16 FAT 表数据

- 相对偏移 0x4~0x5 偏移为第 2 簇 (顺序上第 1 簇)，此处为 FF，表示存储在第 2 簇上的文件 (目录) 是个小文件，只占用 1 个簇便结束了。

- 第 3 簇中存放的数据是 0x0005, 这是一个文件或文件夹的首簇. 其内容为第 5 簇, 就是说接下来的簇位于第 5 簇->FAT 表指引我们到达 FAT 表的第 5 簇指向, 上面写的的数据是"FF FF", 意即此文件已至尾簇.
- 第 4 簇中存放的数据是 0x0006, 这又是一个文件或文件夹的首簇. 其内容为第 6 簇, 就是说接下来的簇位于第 6 簇->FAT 表指引我们到达 FAT 表的第 6 簇指向, 上面写的的数据是 0x0007, 就是说接下来的簇位于第 7 簇->FAT 表指引我们到达 FAT 表的第 7 簇,.... 直到根据 FAT 链读取到扇区相对偏移 0x1A~0x1B, 也就是第 13 簇, 上面写的的数据是 0x000E, 也就是指向第 14 簇->14 簇的内容为"FFFF", 意即此文件已至尾簇.

FAT 表记录了磁盘数据文件的存储链表, 对于数据的读取而言是极其重要的, 以至于 Microsoft 为其开发的 FAT 文件系统下的 FAT 表创建了一份备份, 就是我们看到的 FAT2.FAT2 与 FAT1 的内容通常是即时同步的, 也就是说如果通过正常的系统读写对 FAT1 做了更改, 那么 FAT2 也同样被更新. 如果从这个角度来看, 系统的这个功能在数据恢复时是个天灾.

4.1.4 FAT 根目录

FAT 文件系统的目录结构其实是一颗有向的从根到叶的树, 这里提到的有向是指对于 FAT 分区内的任一文件 (包括文件夹), 均需从根目录寻址来找到. 可以这样认为, 目录存储结构的入口就是根目录. FAT 文件系统根据根目录来寻址其他文件 (包括文件夹), 故而根目录的位置必须在磁盘存取数据之前得以确定. FAT 文件系统就是根据分区的相关 DBR 参数与 DBR 中存放的已经计算好的 FAT 表 (2 份) 的大小来确定的. 格式化以后, 根目录的大小和位置其实都已经确定下来了, 位置紧随 FAT2 之后, 大小通常为 32 个扇区. 根目录之后便是数据区第 2 簇.

FAT 文件系统的一个重要思想是把目录 (文件夹) 当作一个特殊的文件来处理, FAT32 甚至将根目录当作文件处理 (旁: NTFS 将分区参数、安全权限等好多东西抽象为文件更是这个思想的升华), 在 FAT16 中, 虽然根目录地位并不等同于普通的文件或者说是目录, 但其组织形式和普通的目录 (文件夹) 并没有不同. FAT 分区中所有的文件夹 (目录) 文件, 实际上可以看作是一个存放其他文件 (文件夹) 入口参数的数据表. 所以目录的占用空间的大小并不等同于其下所有数据的大小, 但也不等同于 0. 通常是占很小的空间的, 可以看作目录文件是一个简单的二维表文件. 其具体存储原理是, 不管目录文件所占空间为多少簇, 一簇为多少字节. 系统都会以 32 个字节为单位进行目录文件所占簇的分配. 这 32 个字节以确定的偏移来定义本目录下的一个文件 (或文件夹) 的属性, 实际上是一个简单的二维表, 这个 32 字节的定义如表 4 所示.

表 4: FAT16 目录项 32 个字节定义

Offset	Size	Description
0x0~0x7	8	文件名
0x8~0xA	3	扩展名
0xB	1	0x00: 读写 0x01: 只读 0x02: 隐藏 0x04: 系统 0x10: 子目录 0x20: 归档
0xC~0x15	10	系统保留
0x16~0x17	2	文件最近修改时间
0x18~0x19	2	文件最近修改日期
0x1A~0x1B	2	文件的首簇号
0x1C~0x1F	4	文件长度

- 对于短文件名, 系统将文件名分成两部分进行存储, 即主文件名 + 扩展名. 0x0~0x7 字节记录文件的主文件名, 0x8~0xA 记录文件的扩展名, 取文件名中的 ASCII 码值. 不记录主文件名与扩展名之间的"." 主文件名不足 8 个字符以空白符 (20H) 填充, 扩展名不足 3 个字符同样以空白符

(20H) 填充.0x0 偏移处的取值若为 00H, 表明目录项为空; 若为 E5H, 表明目录项曾被使用, 但对应的文件或文件夹已被删除.(这也是误删除后恢复的理论依据). 文件名中的第一个字符若为“.”或“..”表示这个簇记录的是一个子目录的目录项.“.”代表当前目录;“..”代表上级目录 (和我们在 dos 或 windows 中的使用意思是一样的, 如果磁盘数据被破坏, 就可以通过这两个目录项的具体参数推算磁盘的数据区的起始位置, 猜测簇的大小等等, 故而是比较重要的);

- 0xB 的属性字段: 可以看作系统将 0xB 的一个字节分成 8 位, 用其中的一位代表某种属性的有或无. 这样, 一个字节中的 8 位每位取不同的值就能反映各个属性的不同取值了. 如 00000101 就表示这是个文件, 属性是只读、系统.
- 0xC~0x15: 在原 FAT16 的定义中是保留未用的. 在高版本的 WINDOWS 系统中有时也用它来记录修改时间和最近访问时间. 那样其字段的意义和 FAT32 的定义是相同的, 见后边 FAT32.
- 0x16~0x17: 中的时间 = 小时 * 2048 + 分钟 * 32 + 秒 / 2. 得出的结果换算成 16 进制填入即可. 也就是: 0x16 字节的 0~4 位是以 2 秒为单位的量值; 0x16 字节的 5~7 位和 0x17 字节的 0~2 位是分钟; 0x17 字节的 3~7 位是小时.
- 0x18~0x19: 中的日期 = (年份 - 1980) * 512 + 月份 * 32 + 日. 得出的结果换算成 16 进制填入即可. 也就是: 0x18 字节 0~4 位是日期数; 0x18 字节 5~7 位和 0x19 字节 0 位是月份; 0x19 字节的 1~7 位为年号, 原定义中 0~119 分别代表 1980~2099, 目前高版本的 Windows 允许取 0~127, 即年号最大可以到 2107 年.
- 0x1A~0x1B: 存放文件或目录的表示文件的首簇号, 系统根据掌握的首簇号在 FAT 表中找到入口, 然后再跟踪簇链直至簇尾, 同时用 0x1C~0x1F 处字节判定有效性. 就可以完全无误的读取文件 (目录) 了.
- 普通子目录的寻址过程也是通过其父目录中的目录项来指定的, 与数据文件 (指非目录文件) 不同的是目录项偏移 0xB 的第 4 位置 1, 而数据文件为 0.

对于整个 FAT 分区而言, 簇的分配并不完全总是分配干净的. 如一个数据区为 99 个扇区的 FAT 系统, 如果簇的大小设定为 2 扇区, 就会有 1 个扇区无法分配给任何一个簇. 这就是分区的剩余扇区, 位于分区的末尾. 有的系统用最后一个剩余扇区备份本分区的 DBR, 这也是一种好的备份方法. 早的 FAT16 系统并没有长文件名一说, Windows 操作系统已经完全支持在 FAT16 上的长文件名了. FAT16 的长文件名与 FAT32 长文件名的定义是相同的.

4.2 文件在哪里

图15在一个 TF 卡中创建了 test1、test2、test3、test4 四个文件夹和一个 0000.media 媒体文件. System Volume Information 目录及其下面的文件是在 Windows 系统格式化的时候系统写入的系统文件.

字节偏移	字段长度/字节	字段内容及含义	
0x00	8	主文件名	
0x08	3	文件的扩展名	
0x0B	1	文件属性	00000000（读/写）
			00000001（只读）
			00000010（隐藏）
			00000100（系统）
			00001000（卷标）
			00010000（子目录）
			00100000（文件）
0x0C	1	未用	
0x0D	1	文件创建时间精确到 10 ms 的值	
0x0E	2	文件创建时间，包括时、分、秒	
0x10	2	文件创建日期，包括年、月、日	
0x12	2	文件最近访问日期，包括年、月、日	
0x14	2	文件起始簇号的高位	
0x16	2	文件修改时间，包括时、分、秒	
0x18	2	文件修改日期，包括年、月、日	
0x1A	2	文件起始簇号的低位	
0x1C	4	文件大小（以字节为单位）	

图 17: FAT32 目录项定义

根据图17定义可以对根目录下的目录项进行解析, 如图 18所示.

System Volume Information 长文件名项									
001000000	42	20	00	49	00	6E	00	66	00 6F 00 0F 00 72 72 00
001000010	6D	00	61	00	74	00	69	00	6F 00 00 00 6E 00 00 00
001000020	01	53	00	79	00	73	00	74	00 65 00 0F 00 72 6D 00
001000030	20	00	56	00	6F	00	6C	00	75 00 00 00 6D 00 65 00
System Volume Information 短文件名项									
001000040	53	59	53	54	45	4D	7E	31	20 20 20 16 00 05 35 9A
001000050	AF	58	AF	58	00	00	36	9A	AF 58 03 00 00 00 00 00
新建文件夹 长文件名项 (已被删除)									
001000060	E5	B0	65	FA	5E	87	65	F6	4E 39 59 0F 00 75 00 00
001000070	FF	FF	FF	FF	FF	FF	FF	FF	FF FF 00 00 FF FF FF FF
新建文件夹 短文件名项 (已被删除)									
001000080	E5	C2	BD	A8	CE	C4	7E	31	20 20 20 10 00 2B 39 9A
001000090	AF	58	AF	58	00	00	3A	9A	AF 58 06 00 00 00 00 00
test1 目录项									
0010000A0	54	45	53	54	31	20	20	20	20 20 20 10 08 2B 39 9A
0010000B0	AF	58	AF	58	00	00	3A	9A	AF 58 06 00 00 00 00 00
新建文件夹 长文件名项 (已被删除)									
0010000C0	E5	B0	65	FA	5E	87	65	F6	4E 39 59 0F 00 75 00 00
0010000D0	FF	FF	FF	FF	FF	FF	FF	FF	FF FF 00 00 FF FF FF FF
新建文件夹 短文件名项 (已被删除)									
0010000E0	E5	C2	BD	A8	CE	C4	7E	31	20 20 20 10 00 AF 40 9A
0010000F0	AF	58	AF	58	00	00	41	9A	AF 58 07 00 00 00 00 00
test2 目录项									
001000100	54	45	53	54	32	20	20	20	20 20 20 10 08 AF 40 9A
001000110	AF	58	AF	58	00	00	41	9A	AF 58 07 00 00 00 00 00

图 18: FAT32 目录项数据

以 test2 目录项举例, 我们可以看到.

- 文件名为 test2 (短文件名);
- 文件属性为 10 (子目录), 偏移 0x0B;
- 这里时间需要转换, 2 字节用不同的位表示年月日和时分秒;

- 起始簇号为 07 号;
- 如果目录项是以 E5 开头, 那表示该项是无效的或是已经删除了的目录项, 比如上面的四个”新建文件夹”目录项;
- 通过目录项, 我们可以知道存储设备上都有哪些文件和目录, 相应的子目录也是一样的实现, 只不过子目录下面的目录项是在子目录所在的簇中记录;

4.2.2 文件磁盘空间分配

在 FAT32 文件系统中, 它是以簇为单位进行空间分配和管理. 一般一个簇的大小为 4KB(下面均以 4KB 做参考). 一个文件或是一个目录, 它是通过目录项知道它在存储设备上存放的开始位置, 也就是开始簇号, 而簇号信息, 是存储在 FAT 表上, 一个 FAT32 文件系统有两个 FAT 表, 一个正常使用, 另外一个为备份 FAT 表通过分区上的 DBR 和 FSINFO 信息可以知道 FAT 表的大小和所在位置等信息, 具体数据如图 19 所示.

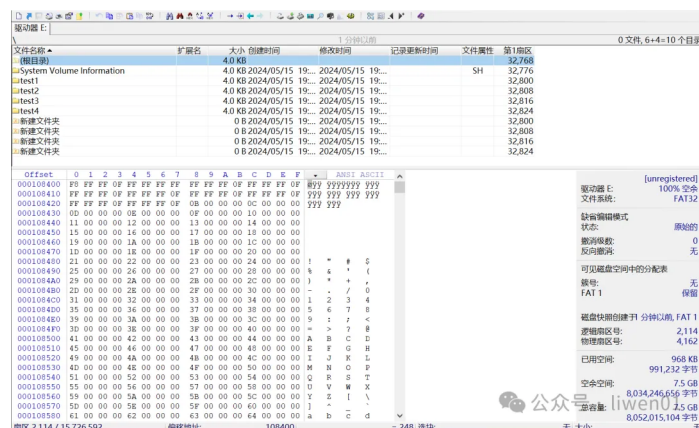


图 19: FAT32 文件簇数据

FAT32 是以 32 位 (4Byte) 来定义一个 FAT 表项, 也就是一个簇的状态, 表 5 是 FAT 表项中值的含义.

表 5: FAT32 簇定义	
FAT 项值 (32 位)	含义
0000 0000H	未使用的簇
0000 0002H~0FFF FFEH	一个已分配的簇号
0FFF FFF0H~0FFF FFF6H	保留
0FFF FFF7H	坏簇
0FFF FFFFH	文件结束簇

对于 FAT 表, 第 0 号簇是固定的 0x0FFFFFFF8, 第 1 号簇项 0xFFFFFFFF 是被系统使用, 第 3 号簇是根目录的开始簇, 如果其值是 0x0FFFFFFF, 表示根目录只占用一个簇的空间, 也就是 4KB 大小空间, 如果其值是 0x00000002~0xFFFFFFFFE, 表示根目录的下一个簇号, 直到出现文件结束簇 0xFFFFFFFF, 也就是根目录大于 4KB 的大小. 图 20 是对 FAT 表现的一个解析.

000108400	0号簇项 F8 FF FF 0F	1号簇项 FF FF FF FF	2号簇项 FF FF FF 0F	3号簇项 FF FF FF 0F
000108410	4号簇项 FF FF FF 0F	5号簇项 FF FF FF 0F	6号簇项 FF FF FF 0F	7号簇项 FF FF FF 0F
000108420	8号簇项 FF FF FF 0F	9号簇项 FF FF FF 0F	10号簇项 0B 00 00 00	11号簇项 0C 00 00 00
000108430	12号簇项 0D 00 00 00	13号簇项 0E 00 00 00	14号簇项 0F 00 00 00	15号簇项 10 00 00 00
000108440	11 00 00 00	12 00 00 00	13 00 00 00	14 00 00 00
.....				
0号簇项	FAT表开始			
1号簇项	系统保留			
2号簇项	根目录			
3号簇项	文件1 -->System Volume Information 目录			
4号簇项	文件2 -->IndexerVolumeGuid 文件			
5号簇项	文件3 -->WPSettings.dat文件			
6号簇项	文件4 -->test1 目录			
7号簇项	文件5 -->test2 目录			
8号簇项	文件6 -->test3 目录			
9号簇项	文件7 -->test4 目录			
10号簇项~242号簇项	文件8 -->0000.media 文件			

图 20: FAT 表解析

从图20可以看出.

- 如果文件或是目录小于 4K(一个簇), 那它所占用的空间就是目录项中起始簇号所分配的空间, 该簇号的值为结束簇号的值 (0x0FFFFFFF);
- 如果一个文件大于 4K(一个簇), 目录项中的起始簇号所在位置的值, 就是下一个位置存储的簇号值, 比如 0000.media 文件, 它的起始簇号是 10, 第 10 簇号 (0x0000000B)-> 第 11 簇号 (0x0000000C)->.....第 241 簇号 (0x000000F2)-> 第 242 簇号 (0x0FFFFFFF 结束簇号);
- 上面这个 0000.media 文件是以连续的方式存储在磁盘中, 当磁盘满了或是使用久了之后, 会存在磁盘碎片, 有可能就不是连续的空间了.

4.3 实现原理

我们从文件的创建、数据写入、文件删除等操作流程看文件系统的基本实现原理

4.3.1 文件创建

- 创建文件或是目录时候, 会先在当前目录所在位置的目录项中添加一个目录项
- 会记录文件的起始簇号, 创建、修改时间, 文件属性等信息

4.3.2 文件增删数据

- 如果起始簇空间写满了, 系统会查找一个空闲簇, 数据将继续写入到该空闲簇中, FAT 表中该空闲簇会被标记已经被使用, 同时, 该文件的结束簇号也会往后移动一个簇.
- 更新该目录项中的修改时间、文件大小等信息
- 删除或是修改文件里面的数据, 就是一个反向的过程

4.3.3 文件数据读取

- 通过目录项, 找需要读取文件所在的开始簇位置
- 如果文件大于一个簇, 开始簇位置的值为下一个簇的位置, 可以顺着这个簇链一直查找, 直到结束簇出现.

4.3.4 文件删除

- 文件删除的时候,根目录中该目录项的信息并不会被删除,而是将该目录项标记为删除状态;
- FAT 表中该文件所占用的簇号,会被标记为 0,表示该簇为未使用的簇.
- 该文件所在簇号所对应扇区的实际文件数据不会被擦除,文件里面的数据还是存储在删除上.

文件删除,实际上也就是将该文件在 FAT 表中的簇信息标记为可使用,然后将目录项标记为已删除,实际数据不会做删除处理

如果要恢复被删除的文件,可以根据目录项中的信息进行恢复,前提是不要再创建新文件和写入新数据,因为新的数据容易将原来文件所在扇区的数据覆盖或是擦除.

4.3.5 基本原理

FAT32 文件系统的基本原理,是通过目录项来管理磁盘的文件目录结构,然后通过 FAT 表来管理磁盘文件所使用的簇(扇区)空间.

FAT32 文件系统的 FAT 表是通过单向链式的方法来管理扇区,这种方式在小文件和小容量的储存设备上使用比较方便,但不适合于大文件和大容量的存储设备.目前大于 32GB 的 SDXC 卡,SD 协会已采用 exFAT 作为默认的文件系统.